

Spring MD

Spring MD: The Ultimate Potential Function Training
and Molecular Dynamics Execution Software

SCX

202671

v1.0 — — SCX — Spring MD

Abstract

SPRING MD——SPRING MDSRINGYAJIE M_t ARBITER `spring-md train``spring-md run``spring-md audit``spring-md compare`

SPRING MD **BORN AUDITED** SPRING M_t CERCISYAJIE——”born audited”MD
SPRING MDtrajectory ARBITER——CERCISYAJIE

SPRING MDNEP/DeepMD/MACE ——

SPRING MDAPI LAMMPS/ASEARBITER SCX Spring——

Spring MDArbiter YajieCercisLAMMPSASE

Contents

1

1.1 DFTMD

Molecular Dynamics, MD MD

1 DFT

2 NEPDeepMDACEGAPMTPDFT $V_{\theta}(\mathbf{R})$

3 MDLAMMPSASE

4 MD——

textbf

- DFTk——

textit

- RMSE——RMSE ”

textit”

- MD——

textit OOD MD

textit

- ——

MDLAMMPSGROMACSAVASP ””trust-me——

1.2 SCXMD

SCX[?, ?, ?, ?]

(1) $M = 1$ ——NEPDeepMDACEGAPMTPSCX2

(2) $M > 1$ SCX1

(3) SCX3

(4) SPRING””Spring Trainer[?]

textbf—

SPRING MD

1.3 Spring MD

SPRING MD

NEP/DeepMD/ACEMDLAMMPS/GROMACS

textbfAudit Middleware—— DFT

SPRING MD

1 **BORN AUDITED** SPRING MD M_t CERCISYAJIE ”born audited” MD

2 SPRING MD ARBITER—— CERCISYAJIE

3 NEP/DeepMD/MACE SPRING MD spring-md train

4 LAMMPS/ASE SPRING MD MD——LAMMPSASE

5 DFT

1.4

- **PES** Potential Energy Surface —— $E(\mathbf{R})$
- **Potential Function** $PES V_\theta(\mathbf{R}) \approx E(\mathbf{R})$
- **Trajectory** MD $\{\mathbf{R}(t_1), \dots, \mathbf{R}(t_T)\}$
- **Audit Report** ARBITER CERCISYAJIE OOD
- **Born Audited** /

1.5

(1) 2——SPRING MD

(2) 3 spring-md train——

(3) 4 spring-md run——

(4) 5 spring-md audit——

(5) 6 spring-md compare——CERCIS

(6) 7 ARBITER——

(7) 8 API——

(8) 9 LAMMPS/ASE

(9) 10

(10) 11 NEP/DeepMD/MACE

(11) 12——

(12) 13

2 Spring MD

2.1

SPRING MDspring-md

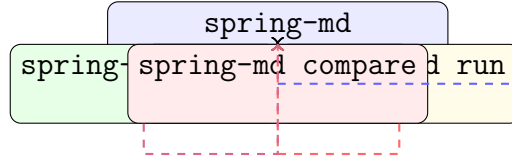


Figure 1: SPRING MD trainrunrunaudit traincompare

2.2

Table 1: SPRING MD

train	DFTtextxyz/npz	+ M_t + CERCIS + YAJIE	SPRING→YAJIE→ M_t →CERCIS
run	+	+ ARBITER	LAMMPS/ASE +
audit	+	ARBITER	YAJIE + OOD
compare	+	CERCIS	

2.3

SPRING MDMD

textbf

1 train SPRING $V_{\text{consensus}}$ M_t CERCIS S_{CERCIS} YAJIES Y_{AJIE} Lyapunov Ψ

2 run → LAMMPS/ASEMD → k ARBITERCERCISYAJIE → " + "

3 audit SPRING MDspring-md audit YAJIEOODMDLAMMPSGROMACS

4 compare spring-md compare CERCIS

Remark 2.1. SPRING MD

textbfAudit Non-Bypassability spring-md runMD

textitARBITER——""flag LAMMPSASE——SPRING MD

3 spring-md train

3.1

Listing 1: spring-md train

```
1 spring-md train [OPTIONS] <INPUT>
2
3 INPUT:
4   <data.xyz>          DFT (extended XYZ)
5   <data.npz>          NPZ ({R, E, F, Z})
6
7 OPTIONS:
8   --config <FILE>      Spring (YAML)
9   --output <DIR>       (: ./spring_output/)
10  --arch <ARCH>         : nep | deepmd | ace | mace (: nep)
11  --M-min <INT>         (: 3)
12  --M-max <INT>         (: 32)
13  --epochs <INT>        (: 1000)
14  --batch-size <INT>    (: 32)
15  --lr <FLOAT>          (: 1e-3)
16  --noise-threshold <FLOAT> Yajie (: 0.05)
17  --device <STR>        : cuda | cpu (: cuda)
18  --seed <INT>          (: 42)
19
20 OUTPUT ( <DIR>/ ):
21   potential.pt          (PyTorch)
22   potential.yaml        (YAML)
23   audit_report.json     Cercis + Yajie + Mt + Lyapunov
24   training_log.csv      (loss, consensus, lyap per epoch)
25   checkpoints/
```

3.2

spring-md trainSPRING

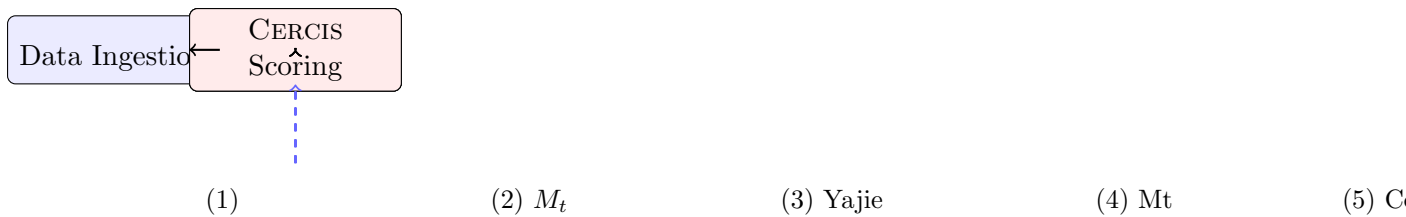


Figure 2: `spring-md train(3)(2)`

1Data Ingestion

DFT

- **extended XYZ**
- **NPZ** NumPyR $N_{\text{data}} \times N_{\text{atoms}} \times 3$ EFZ

- (1)
- (2) $|E| > 10^6 \text{ eV} |F| > 10^4 \text{ eV/\AA}$
- (3) $\Sigma_0(\mathcal{D})(??)M_t$
- (4) $\mathcal{H}(\mathcal{D}) = \text{SHA-256}(\mathcal{D})$
- (5) 80%10%10%

2Multi-Expert Training

SPRING??

Algorithm 1 Spring

Require: $\mathcal{D}_{\text{train}}$ $\mathcal{D}_{\text{val}}$ $\mathcal{D}_{\text{calib}}$

Require: T_{max} $\varepsilon_{\text{conv}}$

Ensure: $V_{\text{consensus}}$

```

1:  $\Psi_0 \leftarrow \infty; t \leftarrow 0$ 
2:  $\mathcal{H}_{\text{data}} \leftarrow \text{SHA-256}(\mathcal{D}_{\text{train}})$ 
3:  $\Sigma_0 \leftarrow \text{ComputeComplexity}(\mathcal{D}_{\text{train}})$ 
4:  $M_0 \leftarrow \Xi(\mathcal{H}_{\text{data}}, \Sigma_0)$ 
5: while  $t < T_{\text{max}}$  and  $\Psi_t > \varepsilon_{\text{conv}}$  do
6:    $\{\mathcal{D}_1, \dots, \mathcal{D}_{M_t}\} \leftarrow \text{StratifiedBootstrap}(\mathcal{D}_{\text{train}}, M_t)$ 
7:   parallel for  $m = 1$  to  $M_t$  do
8:      $\theta_m^{(t)} \leftarrow \text{TrainExpert}(\mathcal{D}_m, \theta_m^{\text{init}})$ 
9:   end parallel for
10:   $\mathbf{s}_{\text{YAJIE}}^{(t)} \leftarrow \text{YajieConsensus}(\{f_{\theta_m}\}_{m=1}^{M_t}, \mathcal{D}_{\text{calib}})$ 
11:   $\Psi_t \leftarrow \frac{1}{|\mathcal{D}_{\text{train}}|} \sum_i \text{Var}_m[f_m(\mathbf{R}_i)]$ 
12:   $\mathcal{D}_{\text{train}} \leftarrow \text{RemoveNoise}(\mathcal{D}_{\text{train}}, \mathbf{s}_{\text{YAJIE}}^{(t)}, \tau_{\text{noise}})$ 
13:   $M_{t+1} \leftarrow \Xi(\mathcal{H}_{\text{data}}, \Psi_t, \mathbf{s}_{\text{YAJIE}}^{(t)}, \Sigma_0)$ 
14:   $t \leftarrow t + 1$ 
15: end while
16:  $V_{\text{consensus}} \leftarrow \sum_m \omega_m f_{\theta_m}$ 
17:  $S_{\text{CERCIS}} \leftarrow \text{CercisScore}(V_{\text{consensus}}, \mathbf{s}_{\text{YAJIE}}, \Psi_t, \mathcal{D}_{\text{val}})$ 
18: return  $(V_{\text{consensus}}, \mathbf{s}_{\text{YAJIE}}, \Psi_t, M_t, S_{\text{CERCIS}})$ 

```

3YajieYajie Denoising

YAJIE[?]
Spring

- $\mathbf{R}_i M_t$

$$s_{\text{YAJIE}}(\mathbf{R}_i) = \exp \left(-\frac{1}{M_t} \sum_{m=1}^{M_t} \left(\frac{\hat{E}_m(\mathbf{R}_i) - \hat{E}_{\text{YAJIE}}(\mathbf{R}_i)}{\sigma_{\text{calib},m}} \right)^2 \right) \quad (1)$$

- τ_{noise} ”/”
- 2

Remark 3.1. YAJIE——

textit

textit $M_t - 11$ ——

4 $M_t M_t$ Binding

M_t SPRING MD””[?]

Definition 3.2 (M_t). $t M_t^{\text{auto}} \mathcal{H}(\mathcal{D}) \text{Lyapunov} \Psi_{t-1} \text{Yajies}_{\text{YAJIE}}^{(t-1)}$

$$M_t^{\text{auto}} = \Xi(\mathcal{H}(\mathcal{D}); \Psi_{t-1}, \mathbf{s}_{\text{YAJIE}}^{(t-1)}, \Sigma_0(\mathcal{D})) \quad (2)$$

M_t — M_t
textitco-emerge

Proposition 3.3 (M_t). $\mathcal{DH}(\mathcal{D})\Sigma_0(\mathcal{D})$ M_t $M_t = 8M = 1$

5CercisCercis Scoring

CERCISPRING MD[?]

$$S_{\text{CERCIS}} = Q + \eta \cdot N \quad (3)$$

- $Q \in [0, 1]$ YAJIELyapunov

$$Q = \frac{1}{2} (\bar{s}_{\text{YAJIE}} + \exp(-\Psi_T/\Psi_0)) \quad (4)$$

- $N \in [0, 1]$

$$N = \min \left(1, \frac{|\mathcal{D}_{\text{eff}}|}{|\mathcal{D}_{\text{train}}|} \cdot \frac{\sigma_E}{\Delta E_{\text{target}}} \right) \quad (5)$$

- $\eta = 0.2$ —

$$S_{\text{CERCIS}} \in [0, 1.2] 1.2$$

3.3

spring-md trainYAML

Listing 2: spring_config.yaml

```

1 # Spring MD
2 train:
3   data:
4     path: "./data/train.xyz"
5     format: "extxyz"
6     train_ratio: 0.8
7     val_ratio: 0.1
8     calib_ratio: 0.1
9
10  model:
11    architecture: "nep"      # nep | deepmd | ace | mace
12    num_neurons: [30, 30]    #
13    cutoff: 6.0              # ( )
14    descriptor_dim: 128      #
15
16  spring:
17    M_min: 3                 #

```

```

18     M_max: 32                #
19     noise_fraction: 0.05    #
20     consensus_threshold: 0.7 # Yajie
21
22     optim:
23         epochs: 1000
24         batch_size: 32
25         learning_rate: 1.0e-3
26         scheduler: "cosine"
27         weight_decay: 1.0e-5
28
29     output:
30         dir: "./spring_output"
31         save_checkpoints: true
32         checkpoint_freq: 50

```

3.4

spring-md trainaudit_report.json

Listing 3: audit_report.json

```

1  {
2    "spring_md_version": "1.0.0",
3    "timestamp": "2026-07-01T12:00:00Z",
4    "data_hash": "a3f8c2...",
5    "complexity": {
6      "Sigma_0": 1.42,
7      "energy_entropy": 3.21,
8      "energy_range": [-78.5, -72.3],
9      "num_species": 2,
10     "num_configurations": 10000
11   },
12   "training": {
13     "M_t": 12,
14     "M_t_history": [8, 12, 12, 12],
15     "total_epochs": 450,
16     "converged": true,
17     "Lyapunov_final": 0.0012,
18     "Lyapunov_initial": 0.0450
19   },
20   "yajie": {

```

```
21     "mean_consensus": 0.92,  
22     "consensus_histogram": [0, 5, 23, 142, ...],  
23     "noise_fraction_removed": 0.032,  
24     "calibration_set_size": 1000  
25 },  
26 "cercis": {  
27     "score": 1.08,  
28     "quality_Q": 0.92,  
29     "coverage_N": 0.80,  
30     "discount_eta": 0.20  
31 }  
32 }
```

4 spring-md run

4.1

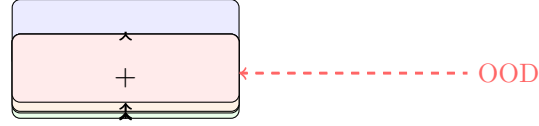
Listing 4: spring-md run

```
1 spring-md run [OPTIONS] <POTENTIAL> <INPUT_STRUCTURE>
2
3 INPUT:
4   <POTENTIAL>          (potential.pt)
5   <INPUT_STRUCTURE>    (XYZ, POSCAR, CIF, LAMMPS data)
6
7 OPTIONS:
8   --config <FILE>      MD (YAML)
9   --output <DIR>       (: ./md_output/)
10  --engine <ENGINE>    MD: lammps | ase (: lammps)
11  --ensemble <ENS>     : nve | nvt | npt (: nvt)
12  --temperature <FLOAT> (K) (: 300)
13  --timestep <FLOAT>   (fs) (: 0.5)
14  --steps <INT>        (: 100000)
15  --audit-every <INT>  (: 100)
16  --device <STR>       (: cuda)
17
18 OUTPUT ( <DIR>/ ):
19   trajectory.xyz       (extended XYZ)
20   trajectory.h5        (HDF5, /)
21   arbiter_report.json  Arbiter
22   md_log.txt           MD
23   thermo.dat           (T, P, E...)
```

4.2

spring-md run

textbfSPRING MDMD—LAMMPSASE



ARBITER

Figure 3: `spring-md runARBITERMDk OODARBITERMD` —

4.3 Audit Hook

`ARBITERMD--audit-every100`

Protocol 4.1 (Arbiter). 1 $\mathbf{R}(t)$ MD

2 $M_t > 1$ `ARBITER` $M_t \mathbf{R}(t)$

- $\text{YAJIE} s_{\text{YAJIE}}(\mathbf{R}(t))$ (??)
- $\sigma_E(t) = \text{std}_m[\hat{E}_m(\mathbf{R}(t))]$
- $\bar{\sigma}_F(t) = \frac{1}{N_{\text{atoms}}} \sum_i \text{std}_m[\hat{F}_{i,m}(\mathbf{R}(t))]$

3 **OOD** $\mathbf{R}(t)$ `ACEACE` $\mathbf{R}(t)$ Mahalanobis95%OOD

4 **Cercis**

$$s_{\text{CERCIS}}^{\text{frame}}(t) = s_{\text{YAJIE}}(\mathbf{R}(t)) \cdot (1 - \mathbb{K}_{\text{OOD}} \cdot 0.5) \quad (6)$$

$\mathbb{K}_{\text{OOD}}$ OODOOD `CERCIS`

5 $t s_{\text{YAJIE}} \sigma_E \bar{\sigma}_F$ OOD $s_{\text{CERCIS}}^{\text{frame}}$ `ARBITER`

6 **OOD** n_{warn} OOD $n_{\text{warn}} = 10$ `ARBITERstderr` —

textit

Remark 4.2 (MD). `ARBITEROOD` `ARBITER`

textit

textitMD OOD `ARBITER` "OOD" —

4.4 MD

Listing 5: `md.config.yaml`

```

1 # Spring MD
2 run:
3   engine: "lammps"          # lammps | ase
4   ensemble: "nvt"           # nve | nvt | npt
5   temperature: 300.0        # (K)

```

```

6  pressure: 1.0          # (bar, NPT)
7  timestep: 0.5          # (fs)
8  total_steps: 100000    #
9
10 thermostat:
11     type: "nose_hoover"  # nose_hoover | berendsen | langevin
12     tau: 100.0          # (fs)
13
14 barostat:               # NPT
15     type: "nose_hoover"
16     tau: 1000.0
17
18 audit:
19     frequency: 100       # ()
20     ood_warn_consecutive: 10 # OOD
21     save_full_experts: false #
22
23 output:
24     dir: "./md_output"
25     format: "xyz"        # xyz | h5 | both
26     thermo_freq: 10      #

```

4.5 LAMMPS

--engine lammepsSPRING MDLAMMPS

- (1) PyTorchLibTorch C++TorchScript LAMMPSpair_styleSPRING MDLAMMPS
pair_stylepair_coeff
- (2) LAMMPS SPRING MDPythonLAMMPS LAMMPSPythonlammeps
- (3) SPRING MDLAMMPS audit-every LAMMPS——SPRING MD
- (4) MD $10 \times \sim 50 \times M_t$ $100M_t = 8 \sim 5\%$ ——

4.6 ASE

--engine aseSPRING MDASEAtomic Simulation EnvironmentPython API

- (1) ASECalculator get_potential_energy()get_forces()
- (2) ASEObserver SPRING MDArbiterObserveraudit-every
- (3) ASELAMMPS——MD IO

5 spring-md audit

5.1

Listing 6: spring-md audit

```

1 spring-md audit [OPTIONS] <TRAJECTORY> [POTENTIAL]
2
3 INPUT:
4   <TRAJECTORY>          (XYZ, H5, LAMMPS dump)
5   [POTENTIAL]           (; )
6
7 OPTIONS:
8   --config <FILE>       (YAML)
9   --output <DIR>        (: ./audit_output/)
10  --M <INT>              ()
11  --ood-method <STR>     OOD: ace_mahalanobis | knn | isolation_forest
12  --per-frame            (: 10)
13  --reference <FILE>
14
15 OUTPUT ( <DIR>/ ):
16   arbiter_report.json   Arbiter
17   per_frame.csv
18   ood_analysis.json     OOD
19   summary.txt

```

5.2

spring-md audit

Table 2: spring-md audit

		-	MD
		/	
+	YAJIE OOD	CERCIS	SPRING MD
+ A + B	A vs B YAJIE		

5.3

spring-md audit SPRING MDV_AV_AV_B DeepMDMACE

Protocol 5.1. 1 $\mathbf{R}(t)V_AV_BE_A(t), E_B(t)\mathbf{F}_A(t), \mathbf{F}_B(t)$

2 $\Delta E(t) = |E_A(t) - E_B(t)|\Delta F(t) = \|\mathbf{F}_A(t) - \mathbf{F}_B(t)\|$

3 ""—— $\Delta E(t) > \varepsilon_E\Delta F(t) > \varepsilon_F$ $\varepsilon_E, \varepsilon_F$ 10 meV/atom0.1 eV/Å

4 ACE

5 "YAJIE"——""

$$s_{\text{YAJIE}}^{\text{cross}}(t) = \exp\left(-\frac{(E_A(t) - E_B(t))^2}{\sigma_A^2 + \sigma_B^2}\right) \quad (7)$$

6

5.4

lspring-md audit

-
-
-
-
- NVENVT

Remark 5.2. ARBITER

textitMD——

MD""

6 spring-md compareCercis

6.1

Listing 7: spring-md compare

```
1 spring-md compare [OPTIONS] <POTENTIALS...> --test-set <FILE>
2
3 INPUT:
4   <POTENTIALS...>      (potential_1.pt potential_2.pt ...)
5   --test-set <FILE>     (extended XYZ)
6
7 OPTIONS:
8   --config <FILE>       (YAML)
9   --output <DIR>        (: ./compare_output/)
10  --metrics <LIST>       : energy, forces, stress, yajie, cercis
11  --per-config           (: )
12  --reference <FILE>     (; "")
13  --plot                 (matplotlib)
14
15 OUTPUT ( <DIR>/ ):
16   ranking.json          Cercis
17   ranking.csv           (CSV)
18   per_config.csv        ( --per-config)
19   comparison_report.pdf ( --plot)
```

6.2

spring-md compare

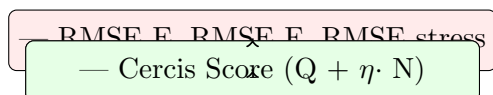


Figure 4: spring-md compareMD SCX//CERCIS

6.3

Algorithm 2 Cercis

Require: $K \{V_k\}_{k=1}^K$ $\mathcal{D}_{\text{test}} = \{\mathbf{R}_i, E_i^{\text{ref}}, \mathbf{F}_i^{\text{ref}}\}_{i=1}^N$
Ensure: Cercis rank_k

```

1: for  $k = 1$  to  $K$  do
2:    $V_k \mathcal{A}_k$ 
3:   //
4:   for  $i = 1$  to  $N$  do
5:      $\hat{E}_i^{(k)} \leftarrow V_k(\mathbf{R}_i); \hat{\mathbf{F}}_i^{(k)} \leftarrow -\nabla V_k(\mathbf{R}_i)$ 
6:   end for
7:    $\text{RMSE}_E^{(k)} \leftarrow \sqrt{\frac{1}{N} \sum_i (\hat{E}_i^{(k)} - E_i^{\text{ref}})^2}$ 
8:    $\text{RMSE}_F^{(k)} \leftarrow \sqrt{\frac{1}{3NN_{\text{atoms}}} \sum_i \|\hat{\mathbf{F}}_i^{(k)} - \mathbf{F}_i^{\text{ref}}\|^2}$ 
9:   //
10:  if  $\mathcal{A}_k$  then
11:     $\text{YAJIE } \{s_{\text{YAJIE}}^{(k)}(\mathbf{R}_i)\}$ 
12:     $\bar{s}_{\text{YAJIE}}^{(k)} \leftarrow \frac{1}{N} \sum_i s_{\text{YAJIE}}^{(k)}(\mathbf{R}_i)$ 
13:     $f_{\text{OOD}}^{(k)} \leftarrow \text{OOD}$ 
14:  else
15:     $\bar{s}_{\text{YAJIE}}^{(k)} \leftarrow \text{NaN}; f_{\text{OOD}}^{(k)} \leftarrow \text{NaN}$ 
16:  end if
17:  // Cercis
18:   $S_{\text{CERCIS}}^{(k)} \leftarrow \text{CercisScore}(V_k, \bar{s}_{\text{YAJIE}}^{(k)}, \text{RMSE}_E^{(k)})$ 
19: end for
20:  $S_{\text{CERCIS}} \rightarrow \text{rank}_k$ 
21: return  $\{(\text{rank}_k, S_{\text{CERCIS}}^{(k)}, \text{RMSE}_E^{(k)}, \bar{s}_{\text{YAJIE}}^{(k)})\}_{k=1}^K$ 

```

6.4 Cercis

Listing 8: ranking.json

```

1 {
2   "test_set": {
3     "path": "./data/test.xyz",
4     "num_configurations": 500,
5     "energy_range": [-80.2, -71.5],
6     "num_species": 2
7   },
8   "rankings": [
9     {
10      "rank": 1,
11      "potential": "spring_nep_M12",
12      "source": "spring-md train",
13      "M_t": 12,

```

```

14     "Cercis_score": 1.08,
15     "RMSE_E": 0.85,
16     "RMSE_F": 52.3,
17     "Yajie_consensus": 0.92,
18     "Lyapunov_final": 0.0012,
19     "OOD_fraction": 0.01
20 },
21 {
22     "rank": 2,
23     "potential": "spring_nep_M8",
24     "source": "spring-md train",
25     "M_t": 8,
26     "Cercis_score": 0.96,
27     "RMSE_E": 0.92,
28     "RMSE_F": 58.1,
29     "Yajie_consensus": 0.87,
30     "Lyapunov_final": 0.0024,
31     "OOD_fraction": 0.03
32 },
33 {
34     "rank": 3,
35     "potential": "deepmd_v2",
36     "source": "DeepMD-kit",
37     "M_t": null,
38     "Cercis_score": null,
39     "RMSE_E": 0.88,
40     "RMSE_F": 55.0,
41     "Yajie_consensus": null,
42     "OOD_fraction": null
43 }
44 ],
45 "notes": ["DeepMDCercisYajie"]
46 }

```

Remark 6.1 (Spring). **NEP/DeepMD/MACEspring-md compare Cercis Spring MD——Spring MD””RMSE ”””””**

7 Arbiter

7.1

ARBITERSPRING MDMD

1 Audit Non-Separability ARBITERtrajectory_hash SHA-256—— ””

2 Per-Frame Traceability MD””——
textit

3 Human-Machine Dual-Readability JSONnotes JSON Schemajq

7.2 Schema

Table 3: ARBITERJSON Schema

report_meta.version	string	SPRING MD
report_meta.timestamp	string	ISO 8601
report_meta.report_type	string	"train" — "run" — "audit"
potential.path	string	
potential.architecture	string	"nep" — "deepmd" — "ace" — "mace"
potential.M.t	int	
potential.data_hash	string	SHA-256
potential.Cercis_score	float	CERCIS
simulation.ensemble	string	"nve" — "nvt" — "npt"
simulation.temperature	float	K
simulation.timestep	float	fs
simulation.total_steps	int	
simulation.engine	string	MD"lammmps" — "ase"
simulation.trajectory_hash	string	SHA-256
audit_frames[N]	array	
audit_frames[].step	int	MD
audit_frames[].time	float	ps
audit_frames[].Yajie_consensus	float	$Y_{AJIE} s_{Y_{AJIE}} \in [0, 1]$
audit_frames[].energy_std	float	σ_E (meV/atom)
audit_frames[].force_std	float	$\bar{\sigma}_F$ (meV/Å)
audit_frames[].is_OOD	bool	OOD
audit_frames[].OOD_score	float	OOMahalanobis/
audit_frames[].Cercis_frame	float	CERCIS
audit_frames[].flags	[string]	"low_consensus" — "high_dispersion" — "critical_ood"
summary.total_frames	int	
summary.mean_consensus	float	YAJIE
summary.consensus_below_0.5	float	0.5
summary.OOD_fraction	float	OOD
summary.Cercis_mean	float	CERCIS
summary.flags_summary	object	flag

7.3

ARBITERsummary

- **Healthy Trajectory** $\text{mean_consensus} > 0.85$ $\text{OOD_fraction} < 0.05$ $\text{Cercis_mean} > 0.80$
- **Warning Trajectory** $\text{mean_consensus} \in [0.5, 0.85]$ $\text{OOD_fraction} \in [0.05, 0.20]$
OOD
- **Unreliable Trajectory** $\text{mean_consensus} < 0.5$ $\text{OOD_fraction} > 0.20$ (a) (b) (c)

8 API

8.1

SPRING MD

(1) `PyTorchpotential.pt torch.save()`

- `state_dict`
- `Mtstate_dict`
-
- `PyTorch_audit_metadata`

(2) `YAMLpotential.yaml`

- `CERCIS`
- `MtYAJIEMt`
-
-

8.2 API

Listing 9: Python API

```
1 from spring_md import SpringPotential
2
3 #
4 potential = SpringPotential.load("spring_output/potential.pt")
5
6 #
7 energy = potential.predict_energy(positions, cell, atomic_numbers)
8 forces = potential.predict_forces(positions, cell, atomic_numbers)
9
10 #
11 expert_energies = potential.predict_energy_all_experts(positions, cell,
12     atomic_numbers)
13
14 #
15 meta = potential.audit_metadata
16 print(f"M_t = {meta['M_t']}")
17 print(f"Cercis = {meta['Cercis_score']}")
18 print(f"Yajie consensus = {meta['Yajie_mean_consensus']}")
```

8.3

SPRING MD

Table 4:

Extended XYZ	.xyz	✓	✓	//
HDF5	.h5, .hdf5	✓	✓	
LAMMPS dump	.dump, .lammppstrj	✓		LAMMPS
ASE trajectory	.traj	✓	✓	ASE
NetCDF	.nc	✓		AMBER/CHARMM

8.4 ArbiterAPI

Listing 10: Python API

```
1 from spring_md import ArbiterReport
2
3 #
4 report = ArbiterReport.load("md_output/arbiter_report.json")
5
6 #
7 print(report.summary())
8 # => "Healthy: 99.2% consensus, 1.3% OOD, Cercis 0.89"
9
10 # OOD
11 ood_frames = report.get_ood_frames(threshold=0.5)
12 for frame in ood_frames:
13     print(f"Step {frame.step}: OOD score = {frame.OOD_score:.2f}")
14
15 #
16 low_consensus = report.get_frames_by_flag("low_consensus")
17
18 #
19 report.plot_consensus_over_time(save="consensus_plot.png")
20
21 # CSV
22 report.export_csv("arbiter_report.csv")
```


9 LAMMPS/ASE

9.1

SPRING MDLAMMPSASE

textbfTransparent Audit Injection

(1) SPRING MDLAMMPSASE

(2) SPRING MDLAMMPSASE/——LAMMPS/ASE SPRING MD

(3) MD

9.2 LAMMPS

Listing 11: LAMMPSPython

```
1 import subprocess
2 from spring_md import SpringPotential, ArbiterHook
3
4 # 1. LAMMPS
5 potential = SpringPotential.load("potential.pt")
6 potential.export_lammps("spring_potential.so") # TorchScript LAMMPS pair style
7
8 # 2. LAMMPS
9 lammps_script = f"""
10 units          metal
11 atom_style     atomic
12 pair_style     spring_md
13 pair_coeff     * * spring_potential.so {element_list}
14
15 read_data      input.data
16 velocity       all create {temp} {seed}
17 fix            nvt all nvt temp {temp} {temp} {tau}
18 thermo         {thermo_freq}
19 thermo_style   custom step temp pe ke etotal press
20 dump           traj all xyz {dump_freq} trajectory.xyz
21 run            {total_steps}
22 """
23
24 with open("in.spring", "w") as f:
25     f.write(lammps_script)
26
27 # 3. LAMMPS
```

```

28 proc = subprocess.Popen(
29     ["lmp", "-in", "in.spring"],
30     stdout=subprocess.PIPE, stderr=subprocess.PIPE
31 )
32
33 # 4. Arbiter
34 hook = ArbiterHook(
35     potential=potential,
36     trajectory_file="trajectory.xyz",
37     audit_every=100,
38     output="arbiter_report.json"
39 )
40 hook.attach_to_process(proc) # ktrajectory.xyz
41
42 proc.wait()
43 hook.finalize() #

```

9.3 ASE

ASEASEPython

Listing 12: ASEPython

```

1 from ase.io import read
2 from ase.md.nvt import NVTBerendsen
3 from ase import units
4 from spring_md import SpringPotential, ArbiterObserver
5
6 # 1. ASE Calculator
7 potential = SpringPotential.load("potential.pt")
8 atoms = read("input.xyz")
9 atoms.calc = potential.to_ase_calculator()
10
11 # 2. MD
12 dyn = NVTBerendsen(
13     atoms,
14     timestep=0.5 * units.fs,
15     temperature=300 * units.kB,
16     taut=100 * units.fs
17 )
18
19 # 3. Arbiter

```

```

20 observer = ArbiterObserver(
21     potential=potential,
22     audit_every=100,
23     output="arbiter_report.json"
24 )
25 dyn.attach(observer, interval=100)
26
27 # 4. MD
28 trajectory_writer = Trajectory("trajectory.traj", "w", atoms)
29 dyn.attach(trajectory_writer.write, interval=10)
30 dyn.run(100000)
31
32 # 5.
33 observer.save_report()

```

9.4

Table 5: Spring MDMD

	LAMMPS	ASE	
NVE	✓	✓	
NVT	✓	✓	
NPT	✓	✓	
	✓	✓	M_t
	✓	✓	
OOD	✓	✓	ACE
	✓	✓	JSON
Langevin	✓	✓	
Nose-Hoover	✓	✓	
	✓	✓	
GPU	✓	✓	PyTorch

10

10.1 1DFT

——DFTMD

```
1 # =====
2 # 1DFT
3 # =====
4 # DFTVASP/CP2K/Quantum ESPRESSOextended XYZ
5 # data/dft_train.xyz (10000)
6
7 # =====
8 # 2
9 # =====
10 spring-md train data/dft_train.xyz \
11     --config configs/train.yaml \
12     --arch nep \
13     --output ./outputs/Si_potential \
14     --device cuda
15
16 # ./outputs/Si_potential/
17 # potential.pt
18 # potential.yaml
19 # audit_report.json      Cercis + Yajie + Mt
20 # training_log.csv
21
22 # =====
23 # 3
24 # =====
25 cat ./outputs/Si_potential/potential.yaml
26 #
27 # Cercis_score: 1.08
28 # M_t: 12
29 # Yajie_mean_consensus: 0.92
30 # Lyapunov_final: 0.0012
31 # Data_hash: a3f8c2e1...
32
33 # =====
34 # 4MDArbiter
35 # =====
36 spring-md run ./outputs/Si_potential/potential.pt data/initial_Si.xyz \
37     --config configs/md.yaml \
```

```

38     --engine lammps \
39     --ensemble nvt \
40     --temperature 300 \
41     --steps 100000 \
42     --audit-every 100 \
43     --output ./outputs/Si_md_run
44
45 # ./outputs/Si_md_run/
46 #   trajectory.xyz
47 #   arbiter_report.json      Arbiter
48 #   thermo.dat
49 #   md_log.txt              MD
50
51 # =====
52 # 5
53 # =====
54 spring-md audit ./outputs/Si_md_run/trajectory.xyz \
55     ./outputs/Si_potential/potential.pt \
56     --output ./outputs/Si_md_audit
57
58 # Python API
59 python -c "
60 from spring_md import ArbiterReport
61 report = ArbiterReport.load('./outputs/Si_md_run/arbiter_report.json')
62 print(report.summary())
63 report.plot_consensus_over_time('consensus.png')
64 ood = report.get_ood_frames()
65 print(f'OOD frames: {len(ood)}/{report.summary_data["total_frames"]}')
66 "
67
68 # =====
69 # 6
70 # =====
71 spring-md compare \
72     ./outputs/Si_potential/potential.pt \
73     ./outputs/Si_potential_alt/potential.pt \
74     ./external/deepmd_Si.pb \
75     --test-set data/test_Si.xyz \
76     --output ./outputs/Si_comparison
77
78 #

```

```
79 cat ./outputs/Si_comparison/ranking.json
```

10.2 2

LAMMPSSPRING MD

```
1 # LAMMPS dumpDeepMD
2 # Spring MD
3
4 # 1
5 spring-md audit legacy_run/dump.lammpstrj \
6     --output ./audit_legacy
7
8 #
9
10 # 2Spring
11 # Spring
12 spring-md audit legacy_run/dump.lammpstrj \
13     ./outputs/Si_potential/potential.pt \
14     --output ./audit_legacy_cross
15
16 # Spring vs DeepMD
17 #
```

10.3 3

```
1 #!/bin/bash
2 #
3
4 MATERIALS=("Si" "SiO2" "SiC" "Si3N4" "Al2O3" "MgO" "TiN" "ZrO2")
5
6 for mat in "${MATERIALS[@]}"; do
7     echo "=== Processing $mat ==="
8
9     #
10    spring-md train data/${mat}_dft.xyz \
11        --output ./ht_outputs/${mat}_potential \
12        --config configs/ht_train.yaml
13
14    # Cercis
15    CERCIS=$(python -c "
```

```

16 import json
17 with open('./ht_outputs/${mat}_potential/audit_report.json') as f:
18     print(json.load(f)['cercis']['score'])
19 ")
20
21 # Cercis > 0.7MD
22 if (( $(echo "$CERCIS > 0.7" | bc -l) )); then
23     spring-md run ./ht_outputs/${mat}_potential/potential.pt \
24         data/${mat}_initial.xyz \
25         --output ./ht_outputs/${mat}_md \
26         --config configs/ht_md.yaml
27 else
28     echo "Skipping $mat: Cercis score $CERCIS < 0.7"
29 fi
30 done
31
32 #
33 spring-md compare ./ht_outputs/*_potential/potential.pt \
34     --test-set data/ht_test.xyz \
35     --output ./ht_outputs/final_ranking

```

11 NEP/DeepMD/MACE

11.1

Table 6: SPRING MDMD

	NEP	DeepMD	ACE	MACE	Spring MD
	✓	✓	✓	✓	✓ ()
=?	55	55	55	55	✓
$M_t > 1$	55	55	55	55	✓
Yajie	55	55	55	55	✓
Cercis	55	55	55	55	✓
LAMMPS	✓	✓		55	✓
ASE	✓	✓	✓	✓	✓
Arbiter	55	55	55	55	✓
OOD	55	55	55	55	✓
	55	55	55	55	✓ (CERCIS)
	55	55	55	55	✓
	55	55	55	55	✓
	55	55	55	55	✓
	55	55	55	55	✓ (SHA-256)

11.2

Definition 11.1. *MD*

textbf

(1) M_t CERCISYAJIE

(2)

(3)

(4)

(5)

??MDNEPDeepMDACEMACEGAPMTP 0/5SPRING MD

Remark 11.2 (—). 0/5SPRING MD5/5

11.3

SPRING MDNEP/DeepMD/ACE

Table 7: NEP1×

NEP ($M = 1$)	1×	1
DeepMD ($M = 1$)	2-3×	1
ACE ($M = 1$)	0.5×	1
MACE ($M = 1$)	3-5×	1
SPRING MD ($M_t = 8$)	6-8×	1 + 8 +
SPRING MD ($M_t = 12$)	9-12×	1 + 12 +

Remark 11.3 (-). $M_t = 8$ SPRING MDNEP6-84-7×

textbf—— M_t /YAJIE "MD"——NEP/DeepMD "MD"—— SPRING MD

12

12.1 Spring MD

SPRING MD

1 YAJIE0.95

textit Schrödinger — DFT

textit

textit

2 $M_t = 86-8 > 10^5 > 10^5$ SPRING MDGPU—

3 **MD** 2%-5% $> 10^6 > 10^7$ $M_t > 16$

4 **Spring** NEP/DeepMD/MACESPRING MD”” spring-md runspring-md compare YAJIECERCIS— SPRING MD

textitspring-md train

5 NEPGPUMDDeepMDDeePMD-kitMACEMACE-MP— SPRING MD

12.2

SPRING MD

- MD ARBITER
- spring-md compareCERCISRMSE
- MD spring-md audit
- – CERCIS3

SPRING MD

textbf

- NEPMACE3-8×
- **ReaxFF** SPRING MD NEPDeepMDACEMACE
- SPRING MD

12.3 SCX

SPRING MDSCX[?, ?, ?, ?, ?] SCX

- **SCX** — $M = 1M > 1$ -

- **Spring**[?] — $\rightarrow\rightarrow\rightarrow\rightarrow$
- **Spring Trainer**[?] — Yajie M_t Cercis
- **Spring MD** — ArbiterLAMMPS/ASE

12.4

- (1) **SPRING MD** ””on-the-fly learning—ARBITEROOD DFTActive Learning
- (2) **GPUSpring** $M_t = 32$
- (3) **Web MDCERCISOOD**
- (4) **SPRING MD DFT**
- (5) **MDARBITER MD**—MD

13

SPRING MD——”” SPRING MDtrainrunauditcompare DFTMD
SPRING MD

- (1) **BORN AUDITED** spring-md train M_t CERCIS YAJIE—— ””
- (2) spring-md run””” + ARBITER” CERCISYAJIEOOD——
textitMD
- (3) SPRING MDspring-md trainNEP/DeepMD/MACE
spring-md auditspring-md compare

**Spring MD”NEP””DeepMD”——
textit MD”-” Spring MD”-”—— NEP/DeepMD/MACE ”auditable by de-
sign”**

SPRING MDv1.0SCX Spring—— SCX1SCX2SCX3- SPRINGYAJIE ”auditable MD”

References

- [1] SCX, “SCX,” *SCX* — , 2026.
- [2] SCX, “SCX,” *SCX* — , 2026.
- [3] SCX, “Yajie,” *SCX* — , 2026.
- [4] SCX, “Spring,” *SCX* — , 2026.
- [5] SCX, “Spring,” *SCX* — , 2026.
- [6] Z. Fan *et al.*, “Neuroevolution machine learning potentials: Combining high accuracy and low cost in atomistic simulations,” *Phys. Rev. B*, vol. 104, p. 104309, 2021.
- [7] L. Zhang, J. Han, H. Wang, R. Car, and W. E, “Deep Potential Molecular Dynamics: A Scalable Model with the Accuracy of Quantum Mechanics,” *Phys. Rev. Lett.*, vol. 120, p. 143001, 2018.
- [8] R. Drautz, “Atomic cluster expansion for accurate and transferable interatomic potentials,” *Phys. Rev. B*, vol. 99, p. 014104, 2019.
- [9] I. Batatia, D. P. Kovács, G. N. C. Simm, C. Ortner, and G. Csányi, “MACE: Higher Order Equivariant Message Passing Neural Networks for Fast and Accurate Force Fields,” *Advances in Neural Information Processing Systems*, 2022.

- [10] A. P. Bartók, M. C. Payne, R. Kondor, and G. Csányi, “Gaussian Approximation Potentials: The Accuracy of Quantum Mechanics, without the Electrons,” *Phys. Rev. Lett.*, vol. 104, p. 136403, 2010.
- [11] S. Plimpton, “Fast Parallel Algorithms for Short-Range Molecular Dynamics,” *J. Comp. Phys.*, vol. 117, pp. 1–19, 1995.
- [12] A. H. Larsen *et al.*, “The Atomic Simulation Environment—a Python library for working with atoms,” *J. Phys.: Condens. Matter*, vol. 29, p. 273002, 2017.

A ArbiterJSON Schema

ARBITERJSON SchemaJSON Schema Draft 2020-12

Listing 13: ArbiterJSON Schema

```

1  {
2    "$schema": "https://json-schema.org/draft/2020-12/schema",
3    "title": "Arbiter Audit Report",
4    "type": "object",
5    "required": ["report_meta", "potential", "simulation", "audit_frames", "summary"],
6    "properties": {
7      "report_meta": {
8        "type": "object",
9        "required": ["version", "timestamp", "report_type"],
10       "properties": {
11         "version": {"type": "string", "pattern": "^\\d+\\.\\d+\\.\\d+$"},
12         "timestamp": {"type": "string", "format": "date-time"},
13         "report_type": {"type": "string", "enum": ["train", "run", "audit"]}
14       }
15     },
16     "potential": {
17       "type": "object",
18       "required": ["path", "architecture", "M_t", "data_hash"],
19       "properties": {
20         "path": {"type": "string"},
21         "architecture": {"type": "string"},
22         "M_t": {"type": "integer", "minimum": 2},
23         "data_hash": {"type": "string", "pattern": "[a-f0-9]{64}"},
24         "Cercis_score": {"type": "number", "minimum": 0, "maximum": 1.2},
25         "Yajie_mean_consensus": {"type": "number", "minimum": 0, "maximum": 1},
26         "Lyapunov_final": {"type": "number", "minimum": 0}
27       }
28     },
29     "simulation": {
30       "type": "object",
31       "required": ["ensemble", "temperature", "timestep", "total_steps"],
32       "properties": {
33         "ensemble": {"type": "string", "enum": ["nve", "nvt", "npt"]},
34         "temperature": {"type": "number", "minimum": 0},
35         "timestep": {"type": "number", "minimum": 0},
36         "total_steps": {"type": "integer", "minimum": 1},
37         "engine": {"type": "string", "enum": ["lammps", "ase"]},
38         "trajectory_hash": {"type": "string"}
39       }
40     },
41     "audit_frames": {
42       "type": "array",
43       "items": {
44         "type": "object",
45         "required": ["step", "time", "Yajie_consensus", "Cercis_frame"],
46         "properties": {
47           "step": {"type": "integer", "minimum": 0},
48           "time": {"type": "number"},
49           "Yajie_consensus": {"type": "number", "minimum": 0, "maximum": 1},
50           "energy_std": {"type": "number", "minimum": 0},
51           "force_std": {"type": "number", "minimum": 0},
52           "is_OOD": {"type": "boolean"},

```

```

53     "OOD_score": {"type": "number", "minimum": 0},
54     "Cercis_frame": {"type": "number", "minimum": 0, "maximum": 1},
55     "flags": {"type": "array", "items": {"type": "string"}}
56   }
57 }
58 },
59 "summary": {
60   "type": "object",
61   "required": ["total_frames", "mean_consensus"],
62   "properties": {
63     "total_frames": {"type": "integer"},
64     "mean_consensus": {"type": "number", "minimum": 0, "maximum": 1},
65     "OOD_fraction": {"type": "number", "minimum": 0, "maximum": 1},
66     "Cercis_mean": {"type": "number", "minimum": 0, "maximum": 1},
67     "flags_summary": {"type": "object"}
68   }
69 }
70 }
71 }

```

B

spring-md

```

1  spring-md <COMMAND> [OPTIONS]
2
3  COMMANDS:
4    train      (Training)
5    run        (MD Simulation)
6    audit      (Trajectory Auditing)
7    compare    (Potential Comparison)
8    help
9
10 GLOBAL OPTIONS:
11   --version
12   --help

```

Table 8: SPRING MD

0	
1	
2	Lyapunov $\Psi_T > \varepsilon_{\text{conv}}$
3	MD
4	
5	GPU/CUDA

C

SPRING MD

```
1 # pip
2 pip install spring-md
3
4 #
5 # - Python >= 3.9
6 # - PyTorch >= 2.0 (with CUDA support for GPU)
7 # - LAMMPS (, lammps)
8 # - ASE >= 3.22 (, ase)
9
10 #
11 spring-md --version
12 # Spring MD v1.0.0 (SCX Audit Middleware)
13 # PyTorch backend: CUDA 12.1
14 # Engines: lammps (found), ase (v3.23.0)
```